Why these KiCad files are not a Z80 computer

Table of Contents

1. What is actually in the directory
2. The prompt was one shot, and the files match that
3. Why the board does not open
4. Why the schematic is a blank sheet
5. The GIF was never a netlist
6. This machine is also missing the libraries
7. How to do the same job so a file can be opened
    7.1. Write the drawing down before drawing it
    7.2. One prompt, one result, one check
    7.3. What to tell the model about the file format
8. Commands that tell you the truth
9. What not to spend the next hour on

This note is about the three KiCad files and the prompt text in this directory. It is not a new design. The checks below were run on this machine with KiCad 10.0.7 (`kicad-cli version` reports `10.0.7`).

The short version: the prompt file was written as one pile of jobs, and the reply sketched three files that **name** a computer. The schematic has no drawings and no wires. The board does not open, and after the one syntax error is removed it still has no pads.

1. What is actually in the directory

File	What it is
`Z80 Simple System Complete prompts for Grok.txt`	Forty lines, one voice, no step that says "stop and open the file". The mailing-list note calls these the revised prompts that were posted with the KiCad files.
`Z80_Simple_Computer.kicad_pro`	A usable project header. Sheet id matches the schematic. `boards` is empty, which is sloppy but not why the board fails to open.
`Z80_Simple_Computer.kicad_sch`	Opens. Draws a blank A3 sheet plus a title block. Seven symbol records, no symbol drawings, no pins, no wires.
`Z80_Simple_Computer.kicad_pcb`	Does not open. Four lines that start with `;` are the reason.
The challenge PDF	The retro-comp post. The picture in it is the real schematic (Thomas Scherrer's "Simple Z80 system" from `http://www.z80.info/gfx/z80test.gif`).

Not in the directory, even though the prompt asks for them: symbol libraries, a netlist, a CSV BOM, a monitor ROM, documentation, a zip, or any rendered image. The post says a monitor program was also generated. Whatever that was, it did not land next to these files.

2. The prompt was one shot, and the files match that

The text file is not a session. It is every demand in a single block: read a GIF, read two more computers, merge "the best aspects", draw a fully wired schematic, route a two-layer board, invent a library per chip, assign footprints, write a monitor, write a manual, and zip it.

Nothing in that list is a check. There is no "open it", no "ERC must pass", no "the netlist must contain these references". A model can finish a reply like that by emitting three short files that talk about the work.

The files do talk about the work, in place of doing it. The schematic title block says the sheet is "fully wired in design intent". The board title block says the traces were "generated conceptually with FreeRouting style". Inside the board file the copper is introduced as an example:

```
; Example tracks (simplified for illustration; full FreeRouting would
fill dense bus)
; More segments and vias would complete the routing of all buses and
control signals
```

Those two sentences are the whole routing job, left as a comment.

The three files also disagree with each other, which is what you get when one reply satisfies several sentences at once:

- The schematic's parts are exactly the parenthetical list in the prompt: `62256`, `27C256`, `8255`, `16C550`, `MAX232`, `74HCT14`, `74HCT32`. The Z80 is not in that list, so the schematic has no Z80.
- The board has a `U1` / `Z80CPU` anyway, because a different sentence said "Z80 computer".
- The title block then claims the schematic "Includes Z80".

- Reference designators do not describe the GIF. On that drawing `U1` is the Z80, `U2` is the RAM and `U3` is the `27C512`. The generated sheet swaps the memories (`U2` is the `27C256`, `U3` is the `62256`) and has no `U1`.

Sending that text one line at a time would have been better only if each line had been a small job with something to open afterwards. Line 11 by itself ("create a Z80 computer … schematic and PCB layout with traces") is already the whole project. Splitting the file on blank lines is not the same as splitting the work.

3. Why the board does not open

KiCad files are s-expressions. They are not Lisp, and `;` does not start a comment. This board has four such lines (the outline remark, the footprint remark, and the two track remarks above).

`pcbnew.LoadBoard` on the file as posted returns no board. `kicad-cli pcb export svg` exits with status 3 and writes nothing. Delete only those four lines and the same `LoadBoard` call succeeds: 8 footprints, 37 net names, 3 copper objects.

That is the entire load failure. The identifiers (`fp-z80`, `edge1`, `seg1`) are not UUIDs, and the file version is the KiCad 8 number (`20240108`) while this KiCad 10 writes `20260206`. Neither of those stopped KiCad 10.0.7 from reading the file once the semicolon lines were gone. Do not put comments in `.kicad_sch` or `.kicad_pcb`. A note belongs in the title block, or as a text item on a user layer.

Opening is not the same as having a board. After the file loads:

- Every footprint has **zero pads**. A line like `Package_DIP:DIP-40_W15.24mm` is only a name. KiCad does not go and fetch the pads when a board is opened. Pads exist only if they were copied into the footprint when it was placed.
- The 37 nets (`GND`, `+5V`, `A0` … `/UART_CS`) are a name table. No pad is a member of any of them, so there is nothing to route.
- The copper is two 10 mm segments and one via. The via sits at `(30, 35)` on `GND`. The segments are at y = 30 (`+5V`) and y = 40 (`GND`). The via touches neither segment and no pad.
- The parts collide. A DIP-40 is about 52 mm long. `U4` is centered at y = 30 and `U5` at y = 80, so those bodies overlap. `U6` and `U8` are 15 mm apart on the same x and also overlap. The DIP-14s centered at y = 140 meet the board edge at y = 150. There is no courtyard in the file, so DRC has nothing to say about it.

FreeRouting cannot repair this. It routes a netlist whose pads already exist. The prompt says "add FreeRouting to the project sandbox". A sentence does not install it, and the board comment admits the traces are conceptual.

4. Why the schematic is a blank sheet

`kicad-cli sch erc` does run, and `kicad-cli sch export svg` does write a plot. The plot is the sheet frame and the title block. Searching that SVG finds `27C256` only inside the title-block comment. The seven symbols are not drawn. The exported netlist has an empty `(components)` list and an empty `(nets)` list.

ERC reports seven warnings and zero errors. Every warning is the same: the footprint library nickname `Package_DIP` is not in the current library table. That warning is real on this machine (see below), and it is also a distraction. A footprint library would not draw these symbols, and it would not create pads that were never stored.

What the schematic file contains is seven records like this, and then the sheet frame:

```
(lib_symbols)
(symbol
  (lib_id "27C256:27C256")
  (at 120.65 76.2 0)
  ...
  (property "Reference" "U2" ...)
  (property "Value" "27C256" ...)
  (property "Footprint" "Package_DIP:DIP-28_W15.24mm" ...)
)
```

From KiCad 6 onward the drawing of a symbol is stored **inside** the schematic, in `(lib_symbols …)`. The `(symbol (lib_id …))` record only places that drawing and holds the reference, value and footprint. Here `(lib_symbols)` is empty, and `27C256:27C256` is not a library that exists anywhere. KiCad therefore has no body, no pins and nothing to put in a netlist.

A placeable symbol also needs, per pin, a `(pin "1" (uuid …))` on the instance, and an `(instances …)` block so the reference is bound to the sheet. None of that was written. There is no `(wire …)`, no label, no power symbol and no junction. "Fully wired in design intent" is the title block describing a schematic that was not drawn.

The same file is the wrong shape for the chips that **are** named. `74HCT14` and `74HCT32` are multi-unit symbols: one inverter or one OR gate per unit, plus a separate power unit. The GIF uses several gates from each package. This sheet places each chip once, as a single unit, with no power pins. That cannot become the decode and reset circuit in the picture.

5. The GIF was never a netlist

The picture in the PDF is a screenshot of an old schematic, not a KiCad file. Reading it is the actual job, and it is the job the prompt skipped.

The drawing is one computer, already built:

- `U1` Z80, 4 MHz oscillator, `74HCT14` used to square the clock
- 32 KiB RAM `62256` at `8000h`–`FFFFh`
- 32 KiB EPROM window `0000h`–`7FFFh`, drawn as a `27C512`
- `82C55` at I/O `00h`–`03h`, `16C550` at I/O `80h`–`87h`
- `MAX232`, RS-232 connector, 1.8432 MHz oscillator for the UART
- reset RC and switch
- `74HCT32` (and the `74HCT14`) doing the address decode, not sitting in a corner as whole-chip boxes
- a 100 nF capacitor at every package

The prompt then says: also study `z80.info/z80_mp.htm` and the Duodyne FirstZ80 tree, and take "the best aspects" of all three. That throws away the one thing the challenge can be graded against. Andrew can diff a KiCad sheet against a computer he has already built. He cannot diff it against a blend of three machines that was never written down.

The `27C256` substitution is a pin change, not a value change. On a 28-pin JEDEC EPROM, pin 1 is `A15` on the `27C512` and `VPP` on the `27C256`. The map in the GIF is already 32 KiB (`0000h`–`7FFFh`), so a `27C256` is big enough, but only if pin 1 is tied the way the 256's datasheet requires and is not left as an address pin. The generated schematic changes the value string and stops. There is no pin 1.

Two further requests fight the file format:

- "Create each symbol in a separate library." KiCad already ships these parts (`CPU`, `Memory_EPROM`, `Memory_RAM`, `Interface`, `74xx`, and `Package_DIP` for the footprints). Placing the stock symbol is what copies a checked drawing into `(lib_symbols)`. A new library per chip is how the sheet ended up pointing at `27C256:27C256`, which is not on disk.
- "Generate a photo-realistic image of the schematic, with no nets drawn on it." An image model will draw a plausible Z80 sheet that is not the file. The picture to keep is the SVG from `kicad-cli`, and it should show the nets.

6. This machine is also missing the libraries

That did not cause the blank sheet or the closed board, and it will cause the next attempt to fail in a new way if it is ignored.

`~/.config/kicad/10.0/sym-lib-table` and `fp-lib-table` only include the default tables under `/usr/share/kicad/template/`. Those two files are not installed here, and there are no `.kicad_sym` or `.pretty` libraries under `/usr/share/kicad`. ERC's `Package_DIP` warning is that fact.

Before any redraw: install the KiCad symbol, footprint and 3D-model packages that match the KiCad version, open KiCad once, and confirm the symbol chooser can find a Z80 and that the footprint chooser can find `Package_DIP:DIP-40_W15.24mm`. Then run the checks in the next section. A library install does not fill in the files that are already here.

7. How to do the same job so a file can be opened

Keep the original GIF as the only source until a sheet loads and ERC is clean. Do not merge in the other two computers in the same pass. Do not ask for the board, the router, the ROM or the manual until the netlist has the parts in the table below.

The person running the tool has to be able to run KiCad, not just receive files. "Add KiCad to the sandbox" is not a step. `kicad-cli` has to be on `PATH`, and every reply that writes a file has to be followed by the commands in Commands that tell you the truth. If `LoadBoard` returns nothing, stop. Do not route a file that did not open.

7.1. Write the drawing down before drawing it

Transcribe the GIF into three tables and correct them by eye against the picture. This is the design. The later prompts are not allowed to invent a net that is not in the tables.

1. Parts: reference, value, footprint, the library id that the symbol chooser actually showed. Include the passives, both oscillators, the reset switch and the connector. Include `U1`.
2. Nets: one row per pin (`U1.30` `A0`, …). Power and no-connects included. For the `27C256` swap, pin 1 is its own row and it is filled in from the datasheet, not copied off the `27C512` pin name.
3. Memory and I/O map, copied off the GIF: ROM `0000h`–`7FFFh`, RAM `8000h`–`FFFFh`, PIO `00h`–`03h`, UART `80h`–`87h`.

Gate symbols get one row per gate (`U7A`, `U7B`, …), not one row per package. The package is decided when the units are assigned.

7.2. One prompt, one result, one check

Each prompt below is a separate turn. Paste the tables with it. Tell it to place stock library symbols and footprints, and not to type a `.kicad_sch` or `.kicad_pcb` from memory. A script that calls KiCad's library code is fine. A hand-written s-expression is how this directory happened: the model has to remember hundreds of pin coordinates, and nothing checks them until the board is dead.

The prompts are deliberately boring.

1. Create an empty KiCad project `Z80_Simple_Computer` with matching schematic and board names. Do not place parts. Stop.

   Check: both files open. The schematic plot is an empty titled sheet. The board opens and has no footprints.

2. Place only `U1` (Z80), its 4 MHz oscillator and the inverter that squares the clock, with power pins tied to `+5V` and `GND` and a 100 nF capacitor on the package. No other IC. Stop.

   Check: those references are in the netlist, the Z80 pin count matches the table, ERC has no "symbol not found" and no unconnected power pin.

3. Place the RAM as `U2` and the `27C256` as `U3` (those are the GIF's designators; only the EPROM value changes), and the decode gates that produce `/ROMCS` and `/RAMCS` for the map above. Wire only those nets. Stop.

   Check: `U3` pin 1 is `VPP` or the tie the table specifies, not `A15`. `U2` is still the RAM. Every address pin in the table appears on exactly the nets named there.

4. Place the `82C55`, the `16C550`, the `MAX232`, the UART oscillator and the connector. Wire I/O decode for `00h`–`03h` and `80h`–`87h`. Stop.

   Check: the netlist component list matches the parts table. No extra IC, no missing `U1`.

5. Assign the through-hole footprints from the table and update the board from the schematic. Draw the edge cuts at most 100 mm by 160 mm (the prompt’s limit; the posted board is 100 by 150 and that difference is not interesting). Place the packages so the courtyards do not overlap and do not cross the edge. Do not route. Stop.

   Check: `LoadBoard` works, every footprint’s pad count equals its pin count, DRC reports unconnected pads rather than a parser failure.

6. Export a Specctra DSN and run FreeRouting, or route in KiCad. Import the result. Do not claim a route in prose. Stop.

   Check: DRC, then a plot of `F.Cu` and `B.Cu`. Unrouted count is zero, or the remaining airwires are listed by net name.

7. Only then: BOM CSV from the schematic, and a monitor ROM whose memory map is the table. The ROM is a text file. It does not belong in the same turn as a footprint.

Refuse two kinds of reply even if the prose sounds finished. A picture of a schematic drawn by an image model is not the sheet. A board file that contains the words "example" or "would complete the routing" is not routed.

7.3. What to tell the model about the file format

These are the constraints that the posted files violated. They are worth pasting into the first prompt so they are not rediscovered the expensive way.

- Do not write `;` comments, `#` notes, or "TODO" sentences inside `.kicad_sch` or `.kicad_pcb`.
- Do not invent a library nickname. Use the id the symbol chooser showed, and after placement the drawing has to be inside `(lib_symbols)`, not only at `(lib_id …)`.
- Do not create one library per chip.
- Place each gate of a `74HCT14` or `74HCT32` as its own unit. Place the power unit and connect it.
- A footprint on the board has to contain its pads. Copy them from the footprint library. Never emit an empty `(footprint "Package_DIP:…")` shell.
- Net names on the board come from the schematic. Do not declare `(net 3 "A0")` by hand and leave it with no pads.
- UUIDs, if a script must write them, are real UUIDs. `fp-z80` happened to be tolerated by 10.0.7 and is still not a UUID.
- Set the file `(version …)` by saving from this KiCad, or by starting from a file this KiCad saved. Do not copy a KiCad 8 skeleton and stamp `generator_version "9.0"` on it. The posted files did that: schematic version `20231120`, board version `20240108`, both claiming generator 9.0. KiCad 10.0.7 can still read those version numbers; the stamp is a sign the file was composed, not saved.

8. Commands that tell you the truth

Run these from the project directory. Exit status matters more than the last paragraph of the reply.

Schematic, plot and netlist:

```
kicad-cli sch erc --severity-all --output erc.rpt
Z80_Simple_Computer.kicad_sch
kicad-cli sch export svg --output plot/ Z80_Simple_Computer.kicad_sch
kicad-cli sch export netlist --output out.net
Z80_Simple_Computer.kicad_sch
```

Then look. `erc.rpt` should name real pins, not seven missing libraries. The SVG should show bodies and wires; if the only hit for `27C256` is the title block, the sheet is the one in this directory. `out.net` should list every reference from the parts table. The posted netlist's `(components)` and `(nets)` are both empty.

Board:

```
python3 - << 'PY'
import pcbnew
b = pcbnew.LoadBoard("Z80_Simple_Computer.kicad_pcb")
assert b is not None, "board did not parse"
fps = list(b.GetFootprints())
print("footprints", len(fps))
for fp in fps:
    print(f"{fp.GetReference():4} {fp.GetValue():12} pads=
{len(list(fp.Pads()))}")
print("tracks", len(list(b.GetTracks())))
PY
kicad-cli pcb drc --output drc.rpt Z80_Simple_Computer.kicad_pcb
```

`LoadBoard` returning nothing means a syntax error, and on the posted file it means the semicolon lines. A footprint with `pads=0` means the library part was named and not copied. DRC on a real update of this circuit should complain about unrouted nets until routing has happened; it should not be the first program that manages to open the file.

There is a longer version of the same rule already written down for the other KiCad experiment, in `/home/falken/_/drv/v/261029-codex-kicad-pcb` (the readable-schematic notes: native ERC, compare the netlist, do not treat a successful tool reply as acceptance). The failure mode in **this** directory is earlier than that. The tool reply produced files that do not contain the circuit.

9. What not to spend the next hour on

Do not hand-repair `Z80_Simple_Computer.kicad_sch` by filling in `(lib_symbols)` from memory. There is no connectivity to preserve. Do not delete the four semicolon lines and start routing: the board that then opens has no pads and its DIP-40s overlap.

Keep the prompt text and these KiCad files as the example of the failure. Start a new project from the pin table. The first success is a schematic plot where `U1` is a Z80 with pins, `U3's EPROM pin 1 matches the `27C256` row, and `out.net` contains the nets from the table. The board comes after that, and FreeRouting comes after the board has pads.